

HOUSEHOLD AUTOPILOT

An Uncertainty-Aware Autonomous Household Decision Agent

Agents for Humans Hackathon — Everyday Agents Track

1. EXECUTIVE SUMMARY

Household Autopilot is an autonomous AI agent that manages routine household purchasing decisions within rules defined by the user.

The system maintains a persistent representation of the household, including household preferences, purchase history, estimated inventory, consumption patterns, budget and purchasing policies.

It operates in the background and performs the repetitive reasoning required to keep the household supplied.

The agent can:

- Monitor household requirements.
- Estimate inventory.
- Predict future needs.
- Understand natural-language household requests.
- Convert activities and recipes into required items.
- Reconcile requirements against estimated household inventory.
- Identify only the items that are actually missing.
- Search and compare available commerce options.
- Select quantities based on household requirements.
- Evaluate the purchase against budget and user policies.
- Automatically act when the decision is sufficiently certain and authorized.
- Ask the user when the decision is uncertain or exceeds the configured authority.
- Explain why it made a decision.
- Update household state after an action.

The core idea is not to build another grocery ordering interface.

The core idea is to build a **household decision layer** that determines whether a household action should happen before using commerce tools to execute it.

The system follows:

UNDERSTAND

↓

```
ESTIMATE
  ↓
CHECK CONFIDENCE
  ↓
REASON
  ↓
CHECK POLICY
  ↓
DECIDE
  ↓
ACT OR ASK
  ↓
UPDATE
  ↓
LEARN
```

2. HACKATHON ALIGNMENT

Household Autopilot is designed for the **Everyday Agents** track of the Agents for Humans Hackathon.

The hackathon requires a new AI agent built with the Strands Agents SDK that performs real work for real people and handles the task end-to-end rather than only chatting about it.

The Everyday Agents track specifically targets daily life, home, money, errands and family, with an emphasis on agents that operate in the background and surface only when a meaningful decision is required.

Household Autopilot directly follows this model.

Instead of requiring the user to repeatedly open an application and manually manage every household purchase, the agent maintains household state and continuously evaluates whether action is required.

The project will use:

- Strands Agents SDK
- Amazon Bedrock
- Amazon Bedrock AgentCore
- AgentCore Runtime
- AgentCore Memory
- AgentCore Gateway
- Amazon DynamoDB
- AWS Lambda
- Amazon EventBridge
- MCP-based commerce integration

AgentCore is particularly relevant because the hackathon explicitly states that AgentCore deployment can strengthen the Technical Implementation score.

3. PROBLEM STATEMENT

Household shopping creates a continuous stream of small decisions.

A person may need to:

- Remember that something is running out.
- Estimate how much remains.
- Decide when to purchase.
- Determine how much to purchase.
- Search for the correct product.
- Compare retailers.
- Compare effective prices.
- Check delivery availability.
- Check whether the product matches their preferences.
- Check whether they can afford it.
- Decide whether the purchase should happen automatically.
- Place the order.
- Track the order.
- Update their understanding of what is available at home.

Each individual task is small.

The problem is that these tasks repeat continuously.

The user therefore becomes the operating system for their own household.

4. THE DEEPER PROBLEM

The problem is not simply:

"How can an AI help me buy groceries?"

The deeper problem is:

"How can an AI agent make routine household decisions on my behalf while knowing when it is confident enough to act and when it should ask me?"

This distinction is central to Household Autopilot.

A shopping assistant primarily responds to a request.

A household agent maintains context before the request exists.

For example:

The user does not say:

"Buy milk."

Instead, the agent observes:

- Historical consumption
- Previous purchases
- Estimated remaining inventory
- Household size
- Current budget
- User's purchase policy

It determines:

"Milk is likely to run out tomorrow."

It then determines:

"Is this prediction reliable enough to act?"

Only after that does it consider purchasing.

5. PROJECT THESIS

The project is built around one principle:

Autonomy should depend on both authorization and confidence.

A user may authorize the agent to automatically purchase groceries up to ₹500.

That does not mean the agent should automatically purchase every grocery item under ₹500.

The agent should also ask:

"How confident am I that this purchase is actually necessary?"

Therefore:

AUTHORIZATION + CONFIDENCE + NECESSITY

↓

AUTONOMOUS ACTION

If authorization exists but confidence is low:

LOW CONFIDENCE

↓

ASK USER

If confidence is high but authorization does not exist:

HIGH CONFIDENCE

+

NO AUTHORIZATION

↓

ASK USER

If the product is not required:

NOT REQUIRED

↓

DO NOTHING

6. WHY UNCERTAINTY MATTERS

Inventory estimation is inherently imperfect.

Purchase history does not provide direct visibility into a physical kitchen.

A household may:

- Consume products faster than expected.
- Consume them slower than expected.
- Share them with guests.
- Waste some quantity.
- Purchase the same product elsewhere.
- Receive a product from someone else.
- Change consumption habits.

Therefore, the system should not represent inventory as absolute truth.

Instead, it should represent:

Estimated Quantity

+

Confidence

+

Last Evidence
+
Prediction

Example:

Milk

Estimated quantity:
1 litre

Estimated consumption:
1 litre/day

Estimated days remaining:
1 day

Confidence:
91%

Decision:
AUTO-PURCHASE

Another example:

Cooking Oil

Estimated quantity:
1.4 litres

Confidence:
42%

Recent evidence:
Conflicting purchases

Decision:
ASK USER

This uncertainty-aware behavior is a core design principle of the project.

7. PRODUCT POSITIONING

Household Autopilot is not positioned as:

- A grocery marketplace.
- A price-comparison application.
- A grocery chatbot.
- A simple auto-reorder system.
- A reminder application.
- A retailer-specific shopping assistant.

It is positioned as:

An autonomous household decision layer that uses commerce tools when action is required.

8. MARKET REALITY

The project does not claim that agentic grocery ordering is a new invention.

Current commerce platforms already expose agentic capabilities.

Swiggy's Builders Club provides MCP access to its commerce platform, including Instamart, with tools for grocery discovery, cart management, checkout and order tracking. Swiggy also explicitly lists Auto-Restock as one of the patterns developers can build.

Swiggy's current MCP documentation also supports UPI-based payment flows, although the project's MVP will use the safest available authorization model and will not treat payment execution as a problem to solve from scratch.

Therefore, the project does not attempt to win by claiming:

"AI can order groceries."

Instead, it focuses on:

What should the household agent decide before it orders anything?

This distinction makes the project more technically defensible and better aligned with the agentic nature of the hackathon.

9. PRIMARY DIFFERENTIATION

The project's main differentiator is the **Household Decision Engine**.

The engine combines:

```
HOUSEHOLD MEMORY
      +
INVENTORY ESTIMATION
      +
CONSUMPTION
      +
USER INTENT
      +
REQUIREMENT GENERATION
      +
BUDGET
      +
POLICY
      +
CONFIDENCE
      +
COMMERCE TOOLS
      ↓
HOUSEHOLD DECISION
```

The agent is therefore not optimized simply to complete more transactions.

It is optimized to make the correct household decision.

10. THE HOUSEHOLD DECISION ENGINE

The Household Decision Engine determines:

1. What is happening?
2. What does the household probably have?
3. What does the household probably need?
4. How confident is the estimate?
5. What does the user want to accomplish?
6. What items are actually missing?
7. What options are available?
8. Which option best matches the household?
9. Is the purchase within budget?
10. Is the purchase authorized?
11. Is the decision sufficiently certain?
12. Should the agent act, wait, or ask?

11. DECISION STATES

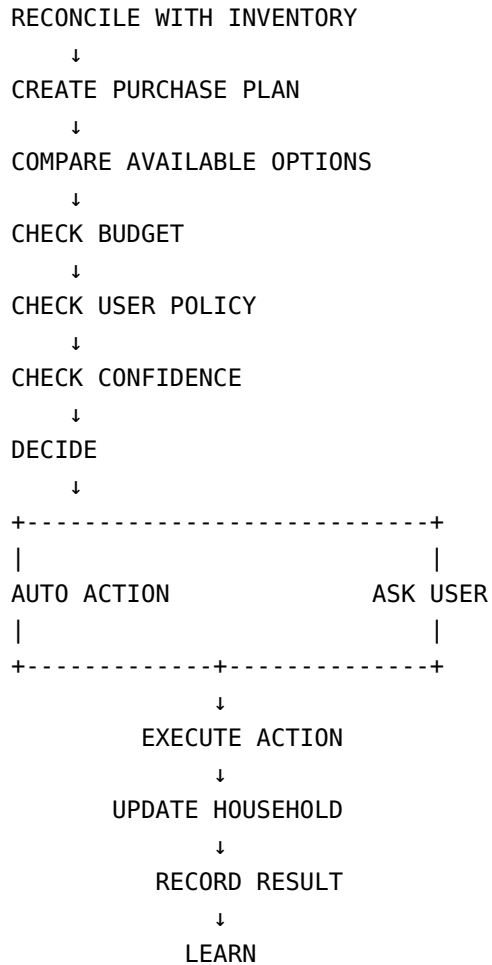
The agent can produce the following decisions:

```
BUY  
  
WAIT  
  
DO_NOT_BUY  
  
ASK_USER  
  
REQUIRE_CLARIFICATION  
  
SWITCH_RETAILER  
  
CHANGE_QUANTITY  
  
CHANGE_PRODUCT  
  
BLOCK_TRANSACTION
```

This makes the agent a decision system rather than a linear automation.

12. CORE AGENTIC LOOP

```
OBSERVE  
  ↓  
RETRIEVE HOUSEHOLD STATE  
  ↓  
UNDERSTAND CURRENT CONTEXT  
  ↓  
ESTIMATE INVENTORY  
  ↓  
CALCULATE CONFIDENCE  
  ↓  
PREDICT REQUIREMENTS  
  ↓  
OR RECEIVE USER INTENT  
  ↓  
INTERPRET INTENT  
  ↓  
GENERATE REQUIREMENTS  
  ↓
```



13. HOUSEHOLD MEMORY

The system maintains persistent household memory.

It stores:

- Household size
- Product preferences
- Brand preferences
- Retailer preferences
- Purchase history
- Consumption patterns
- Inventory estimates
- User corrections
- User approvals
- User rejections
- Budget information
- Purchase policies

- Important household events

The purpose is not simply to remember conversations.

The purpose is to maintain an evolving representation of the household.

14. MEMORY ARCHITECTURE

The memory system can be separated into:

SHORT-TERM CONTEXT

Current conversation and active task.

Example:

"I want to make pasta tonight."

LONG-TERM HOUSEHOLD MEMORY

Persistent information.

Example:

"Household prefers Brand A."

STATE

Current operational information.

Example:

"Milk estimated inventory = 1 litre."

AgentCore Memory is suitable for this architecture because AWS provides persistent memory capabilities for context across sessions, including user-preference and other long-term memory strategies.

15. INVENTORY MODEL

Inventory should not be treated as a deterministic database value.

Instead:

```
Inventory Estimate =  
Historical Purchases  
+  
Consumption Model  
+  
Recent Evidence  
+  
User Corrections  
+  
Household Events
```

Each estimate includes:

```
quantity  
confidence  
timestamp  
consumption_rate  
days_remaining  
evidence
```

16. CONSUMPTION ENGINE

The consumption engine learns how quickly products are normally consumed.

Example:

Previous purchases:

```
Day 1 → 1L milk  
Day 2 → 1L milk  
Day 3 → 1L milk  
Day 4 → 1L milk
```

The agent estimates:

```
Average consumption ≈ 1L/day
```

If the household changes behavior:

New consumption $\approx$ 1.5L/day

The model adapts.

17. CONFIDENCE ENGINE

The Confidence Engine evaluates the reliability of an inventory prediction.

Possible factors:

- Age of purchase data
- Number of historical observations
- Consumption consistency
- Conflicting purchase records
- User corrections
- Household size changes
- Recent events
- Unusual consumption

Example:

Milk

Inventory confidence:
92%

Reason:
Stable daily consumption
Recent purchase data
No conflicting purchases

Another example:

Oil

Inventory confidence:
39%

Reason:
Inconsistent consumption
Recent household event
No recent inventory confirmation

The second situation should not trigger autonomous purchasing.

18. CONFIDENCE-AWARE AUTONOMY

The agent can use configurable confidence thresholds.

Example:

```
Confidence > 85%  
+  
Within policy  
+  
Essential product  
=  
AUTO ACTION
```

```
Confidence 50–85%  
=  
ASK / VERIFY
```

```
Confidence < 50%  
=  
DO NOT AUTO-PURCHASE
```

These thresholds can be adjusted by the application.

This creates a direct connection between uncertainty and agent autonomy.

19. PREDICTIVE REPLENISHMENT

The agent periodically checks whether products are approaching depletion.

Example:

```
Milk  
  
Estimated remaining:  
1 litre  
  
Consumption:  
1 litre/day
```

Predicted depletion:
Tomorrow

Confidence:
91%

The system creates a purchase candidate.

It then evaluates:

- Product
- Quantity
- Retailer
- Price
- Delivery
- Budget
- Policy
- Confidence

20. DO-NOTHING DECISION

The agent must be capable of deciding that no action is necessary.

Example:

Oil

Estimated remaining:
2.3 litres

Expected consumption:
0.8 litres/month

Predicted remaining duration:
~2.8 months

Decision:
DO_NOT_BUY

The system records the decision.

This prevents the agent from becoming a simple recurring-order automation.

21. USER INTENT

The agent also operates reactively.

The user can provide a goal instead of a product list.

Examples:

"I want to make Maggi."

"I want to make pasta."

"I want to make biryani for six people."

"I need breakfast."

"My friends are coming tonight."

The agent converts the goal into a structured requirement.

22. INTENT → REQUIREMENT GENERATION

Example:

USER:

"I want to make pasta for four people."

Agent interpretation:

Intent:

Meal preparation

Meal:

Pasta

People:

4

Time:

Tonight

Requirement generation:

Pasta
Tomato sauce
Cheese
Oil
Spices

The agent then checks household inventory.

23. INTENT → PANTRY RECONCILIATION

Example:

Required:

Pasta
Sauce
Cheese
Oil
Spices

Inventory:

Pasta → Available
Oil → Available
Spices → Available
Sauce → Missing
Cheese → Missing

The resulting purchase plan is:

BUY:

Sauce
Cheese

The agent does not purchase:

Pasta
Oil
Spices

because they are already estimated to be available.

24. MAGGI DEMONSTRATION

The user says:

"I want to make Maggi."

The agent identifies:

Maggi noodles
Oil
Spices
Optional vegetables

The inventory engine returns:

Oil → Available
Spices → Available
Maggi → Missing

The agent creates:

Purchase requirement:
Maggi noodles

It then:

1. Searches the commerce tool.
2. Compares products.
3. Checks availability.
4. Calculates effective cost.
5. Checks budget.
6. Checks policy.
7. Checks confidence.
8. Purchases or asks.

25. WHY THE MAGGI FLOW MATTERS

The important behavior is not:

"AI ordered Maggi."

The important behavior is:

```
USER INTENT
  ↓
UNDERSTAND ACTIVITY
  ↓
GENERATE REQUIREMENTS
  ↓
CHECK HOUSEHOLD STATE
  ↓
REMOVE ALREADY AVAILABLE ITEMS
  ↓
IDENTIFY MISSING ITEMS
  ↓
PLAN PURCHASE
  ↓
CHECK AUTHORITY
  ↓
ACT
```

This demonstrates the agent reasoning about the household rather than simply responding to a product query.

26. MULTI-PERSON INTENT

Example:

"I want to make biryani for six people."

The agent considers:

- Number of people
- Recipe requirements
- Existing inventory
- Package sizes
- Quantity requirements
- Budget
- User preferences

The output is a structured shopping plan.

The system should not require the user to manually create an ingredient list.

27. HOUSEHOLD EVENTS

Temporary events can change consumption.

Examples:

- Guests
- Parties
- Festivals
- Special meals
- Travel
- Temporary household members

Example:

"Five friends are coming tonight."

The agent can create an event context:

```
EVENT:
Guests

PEOPLE:
5

DURATION:
Tonight

EXPECTED IMPACT:
Higher snack and beverage requirements
```

The event is incorporated into future decisions.

28. BUDGET ENGINE

The household provides a monthly spending limit.

Example:

Monthly budget:
₹5,000

Spent:
₹3,800

Remaining:
₹1,200

The budget is a constraint.

It is not a spending target.

The agent should never interpret:

Budget = ₹5,000

as:

Spend ₹5,000.

Instead:

Fulfill necessary household requirements
while minimizing unnecessary spending
within the user's financial constraints.

29. BUDGET-AWARE PLANNING

The agent should consider future expected purchases.

Example:

Monthly budget:
₹5,000

Current spending:
₹4,100

Remaining:

₹900

Predicted essential requirements:

₹750

Expected remaining:

₹150

The system can continue normal operation.

If:

Predicted essential requirements:

₹1,200

the agent should identify a future budget problem and notify the user.

30. PURCHASE POLICY

The user defines explicit autonomy rules.

Example:

Monthly budget:

₹5,000

Automatic transaction limit:

₹500

Allowed categories:

Groceries

Household supplies

Above ₹500:

Ask user

Unknown category:

Ask user

Budget exceeded:

Block

Autonomous shopping:
Enabled

31. AUTHORIZATION VS CONFIDENCE

Two independent conditions control autonomy.

AUTHORIZATION

"Am I allowed to spend this money?"

CONFIDENCE

"Am I sufficiently confident that this purchase is necessary and correct?"

Only when both conditions are satisfied should the agent automatically act.

```
AUTHORIZATION = YES  
CONFIDENCE = HIGH  
NECESSITY = HIGH  
    ↓  
AUTO ACTION
```

Otherwise:

ASK USER

This is a fundamental safety property of the system.

32. RETAILER INTEGRATION

The project will use one primary real commerce integration for the MVP.

The preferred first integration is **Swiggy Instamart through its official MCP platform**, because Swiggy currently provides developer-facing MCP infrastructure with Instamart search, cart, checkout and tracking capabilities.

The system will not attempt to build three complete retailer integrations during the six-week development period.

Additional retailers will be represented through an abstraction layer and can be added later.

33. RETAILER ABSTRACTION

The Household Decision Engine should not depend directly on Swiggy.

Instead:

```
HOUSEHOLD DECISION ENGINE
  ↓
SHOPPING INTERFACE
  ↓
SWIGGY ADAPTER
  ↓
SWIGGY MCP
```

Future:

```
SHOPPING INTERFACE
|
+-- Swiggy
|
+-- Zepto
|
+-- Amazon
|
+-- Other retailers
```

This allows the household intelligence layer to remain independent from any individual commerce provider.

34. COMMERCE TOOL INTERFACE

The internal interface can expose:

```
search_products()
get_product()
check_availability()
create_cart()
get_cart()
checkout()
track_order()
```

The agent does not need to understand the implementation details of each retailer.

It operates against a normalized interface.

35. REAL VS SIMULATED CAPABILITIES

The MVP will distinguish clearly between:

REAL

Capabilities available through an authorized commerce integration.

SIMULATED

Capabilities that are demonstrated through a local/mock adapter because the required external capability is unavailable or unsuitable for the hackathon environment.

The project will never present a simulated order as a real transaction.

This keeps the demonstration technically honest and reproducible.

36. STRANDS AGENTS SDK

Strands Agents SDK will be the core agent framework.

The Strands agent will be responsible for:

- Understanding user intent
- Planning
- Selecting tools
- Reasoning over household context
- Coordinating multi-step tasks
- Determining when clarification is needed
- Producing an action plan

The tools themselves will perform deterministic operations.

Architecture:

STRANDS AGENT

↓

REASONING / PLANNING

↓

TOOLS

37. LLM RESPONSIBILITY

The LLM should handle:

- Natural-language understanding
- Intent interpretation
- Requirement generation
- Reasoning
- Tool selection
- Decision explanation
- Clarification

The LLM should NOT directly control:

- Budget state
- Transaction authorization
- Inventory database writes
- Payment authorization
- Policy overrides

38. DETERMINISTIC CONTROL LAYER

Critical operations are handled by deterministic services.

Example:

```
Agent:
"I recommend purchasing milk for ₹64."

Policy Engine:
₹64 <= ₹500 automatic limit

Budget Engine:
₹64 <= remaining budget

Confidence Engine:
91% confidence

Decision:
AUTHORIZED
```

Only then:

```
checkout()
```

is allowed.

39. TOOL ARCHITECTURE

Potential tools:

```
get_household_state()
get_preferences()
get_inventory()
get_purchase_history()
get_consumption_history()
estimate_inventory()
calculate_inventory_confidence()
predict_depletion()
get_budget_status()
get_purchase_policy()
generate_requirements()
reconcile_inventory()
search_products()
compare_products()
calculate_effective_cost()
check_authorization()
request_confirmation()
```

```
create_cart()

checkout()

track_order()

update_inventory()

update_budget()

record_decision()

update_memory()
```

40. AGENTCORE RUNTIME

The Strands agent will be deployed using Amazon Bedrock AgentCore Runtime.

AgentCore Runtime supports Strands Agents and provides managed runtime capabilities including session isolation and support for asynchronous agent workloads.

The MVP deployment will demonstrate:

```
Strands Agent
  ↓
AgentCore Runtime
  ↓
Household Tools
  ↓
Commerce MCP
```

41. AGENTCORE MEMORY

AgentCore Memory will provide persistent household memory.

The MVP will use it for:

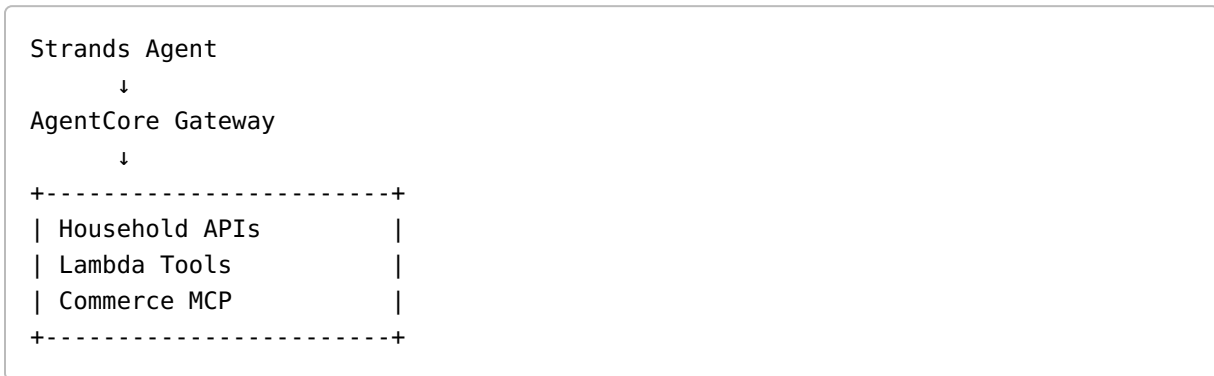
- User preferences
- Household preferences
- Learned shopping behavior
- Relevant long-term context

AWS documentation specifically provides Strands integration with AgentCore Memory and long-term memory strategies.

42. AGENTCORE GATEWAY

AgentCore Gateway will provide the tool-access layer.

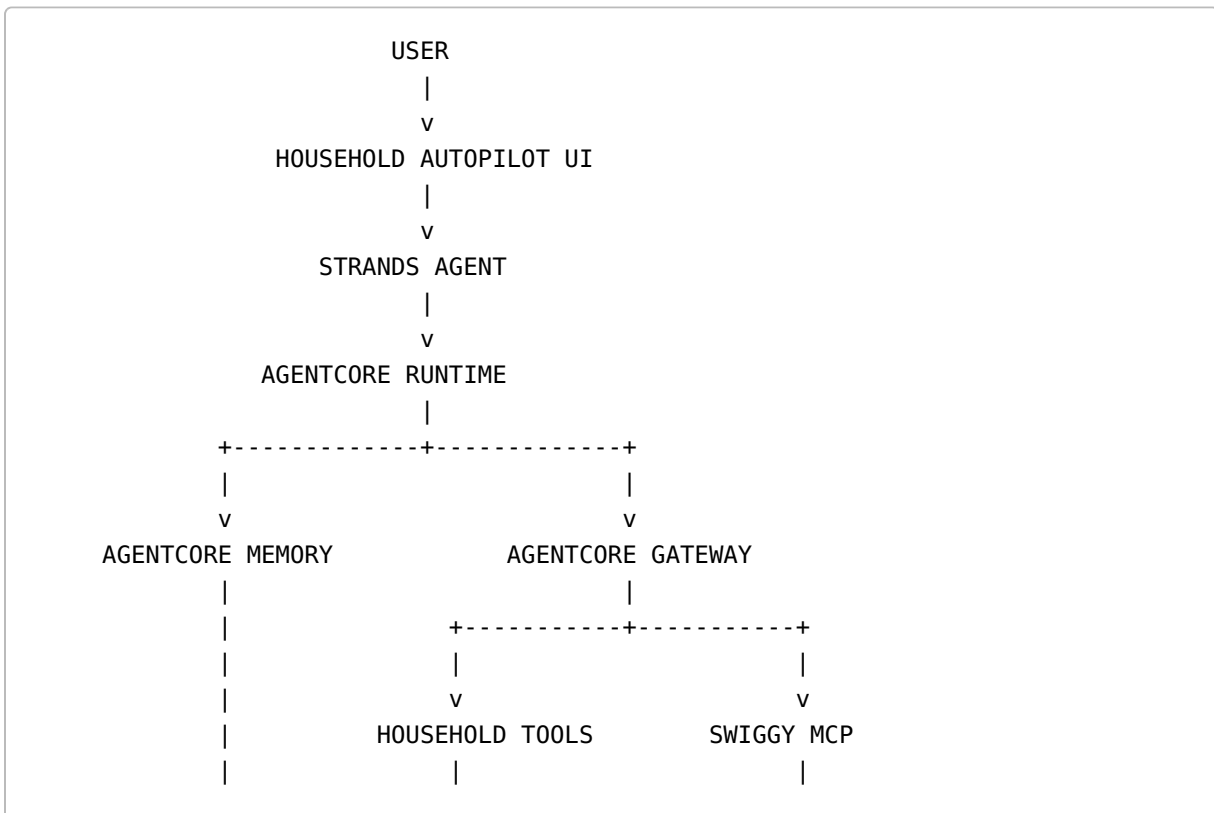
Conceptually:

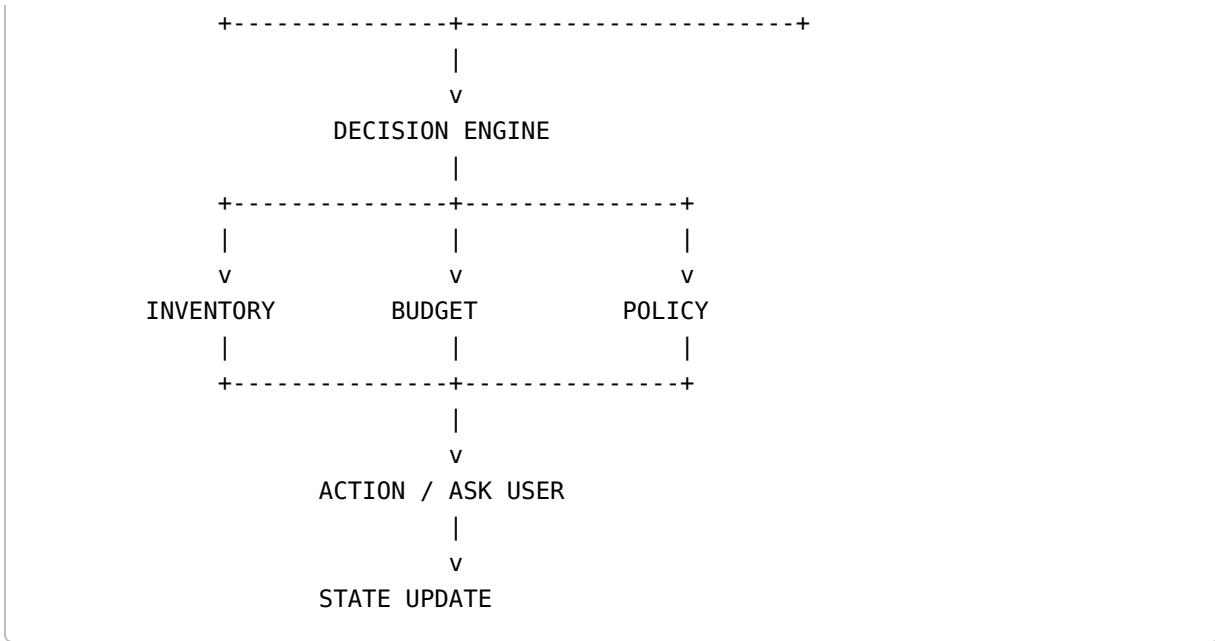


AgentCore Gateway can expose APIs, Lambda functions and existing MCP servers as agent-accessible tools.

43. MVP AWS ARCHITECTURE

The actual six-week implementation will intentionally remain compact.





44. SUPPORTING AWS SERVICES

The MVP will use only the services that materially contribute to the demonstration.

Amazon Bedrock

For foundation model inference and reasoning.

Strands Agents SDK

For agent orchestration.

AgentCore Runtime

For agent deployment.

AgentCore Memory

For persistent household context.

AgentCore Gateway

For tool integration.

DynamoDB

For structured household state.

Lambda

For deterministic business logic and tools.

EventBridge

For one background household evaluation workflow.

CloudWatch

For logs and observability.

45. SERVICES NOT REQUIRED FOR MVP

The following are deliberately not required for the first version:

- Complex multi-agent architecture
- Large-scale Step Functions workflows
- Full AgentCore Identity implementation
- Large evaluation infrastructure
- Multiple real retailer integrations
- Complex notification infrastructure
- Full household subscription management
- Full payment infrastructure
- Full production-grade mobile application

These can be future extensions.

The purpose is to maximize depth of the core agent rather than breadth of the architecture.

46. BACKGROUND EXECUTION

The hackathon specifically emphasizes agents that operate in the background and surface when a meaningful decision is required.

The MVP will implement one scheduled workflow.

Example:

```
EventBridge
  ↓
Household Check
  ↓
Retrieve State
  ↓
Estimate Inventory
  ↓
Calculate Confidence
  ↓
Predict Depletion
  ↓
Decision
```

If nothing is required:

STOP

If a purchase is required:

CONTINUE

This provides a real autonomous background loop without building an unnecessarily complex distributed system.

47. PRIMARY MVP DEMO

The project will focus on exactly three core demonstrations.

DEMO 1 — PREDICTIVE ACTION

The agent predicts that milk will run out.

It checks:

- Inventory
- Confidence
- Budget
- Policy

It automatically performs an authorized purchase.

DEMO 2 — RESTRAINT

The agent evaluates cooking oil.

It determines that sufficient inventory exists.

It does nothing.

This demonstrates that the agent is capable of choosing no action.

DEMO 3 — INTENT TO SHOPPING

The user says:

"I want to make Maggi tonight."

The agent:

- Understands the intent.
- Generates requirements.
- Checks inventory.
- Identifies missing items.
- Searches the commerce tool.
- Checks budget.
- Checks policy.
- Purchases or asks.

These three demonstrations are sufficient to communicate the core product.

48. DEMO 1 — PREDICTIVE PURCHASE

Initial state:

Household:

4 people

Monthly budget:

₹5,000

Milk consumption:

1L/day

Estimated milk:

1L

Confidence:

92%

Auto purchase limit:

₹500

The agent predicts:

Milk will run out tomorrow.

The agent checks the retailer.

Example:

Milk:

₹64

Policy:

₹64 < ₹500

Confidence:

92%

Decision:

AUTO-PURCHASE

The agent executes the authorized order.

49. DEMO 2 — DO NOTHING

The agent checks:

Cooking oil:

2.4L estimated

Consumption:

0.8L/month

Expected remaining:
~3 months

Decision:

DO_NOT_BUY

The UI displays:

No purchase required.

Estimated inventory:
2.4L

Expected remaining:
~3 months

This demonstrates agent restraint.

50. DEMO 3 — MAGGI

User:

"I want to make Maggi tonight."

Agent:

Intent:
Meal preparation

Potential requirements:
Maggi
Oil
Spices

Inventory:

Oil:
Available

Spices:
Available

Maggi:
Missing

Purchase plan:

Maggi

Retailer:

Best available option

Policy:

Within automatic limit

Decision:

AUTO-PURCHASE

After execution:

Inventory updated
Budget updated
Purchase recorded

If confidence or authorization is insufficient:

ASK USER

51. WHY THESE THREE DEMOS

The three demos communicate three different agent capabilities:

PREDICTIVE PURCHASE

=

Agent can act without being prompted.

DO NOTHING

=

Agent can decide not to act.

INTENT SHOPPING

=

Agent understands goals rather than only products.

Together:

OBSERVE

+

REASON

+

DECIDE

+

ACT

+

RESTRAIN

52. USER EXPERIENCE

The product interface will focus on decisions rather than overwhelming the user with agent activity.

HOME

Display:

- Household status
- Budget
- Upcoming requirements
- Active orders
- Important decisions

INVENTORY

Display:

- Product
- Estimated quantity
- Days remaining
- Confidence

APPROVALS

Display only decisions requiring user intervention.

AGENT ACTIVITY

Show meaningful actions:

Milk predicted to run out.

Inventory confidence: 92%.

Best option found.

Purchase authorized.

Order placed.

53. APPROVAL INTERFACE

Example:

RICE REPLENISHMENT

Estimated inventory:
Low

Recommended quantity:
5kg

Best option:
₹1,200

Automatic limit:

₹500

Reason:

Purchase exceeds automatic authorization.

[APPROVE]

[CHANGE]

[CANCEL]

54. EXPLAINABILITY

Every autonomous action should be explainable.

Example:

WHY DID YOU BUY MILK?

Milk was predicted to run out tomorrow.

The inventory estimate had 92% confidence.

The selected product matched the household's preferred brand.

The total cost was ₹64.

Your automatic purchase limit is ₹500.

The purchase was therefore authorized.

55. UNCERTAINTY EXPLANATION

Example:

WHY DID YOU ASK ME?

Your estimated cooking-oil inventory is uncertain.

Confidence:

42%

Recent purchase data conflicts with the consumption estimate.

I did not automatically purchase oil because the inventory estimate was not reliable enough.

This is important because the agent should be able to explain not only why it acted, but also why it refused to act.

56. SAFETY ARCHITECTURE

The architecture is:

```
USER POLICY
  ↓
AGENT RECOMMENDATION
  ↓
CONFIDENCE CHECK
  ↓
BUDGET CHECK
  ↓
AUTHORIZATION CHECK
  ↓
TRANSACTION
```

The LLM cannot bypass these layers.

57. KILL SWITCH

The user can disable autonomous purchasing.

```
AUTONOMOUS ACTIONS
[ ON ]
```

When disabled:

- Monitoring continues.
- Predictions continue.
- Recommendations continue.
- Automatic transactions stop.

58. DUPLICATE PURCHASE PROTECTION

Before an order is placed, the system checks:

- Existing orders
- Recent purchases
- Pending carts
- Current inventory
- Recent agent decisions

This prevents the background workflow from accidentally purchasing the same item repeatedly.

59. DATA MODEL

The MVP can use DynamoDB tables or logical entities for:

```
Household
Preferences
Inventory
Consumption
Purchases
Budget
Policies
Decisions
Orders
Events
```

Example:

```
{
  "householdId": "HH001",
  "monthlyBudget": 5000,
  "spent": 3800,
  "autoPurchaseLimit": 500,
  "autonomousMode": true
}
```

60. INVENTORY RECORD

Example:

```
{
  "product": "milk",
  "estimatedQuantity": 1,
  "unit": "litre",
  "consumptionRate": 1,
  "estimatedDaysRemaining": 1,
  "confidence": 0.92,
  "lastUpdated": "2026-09-10"
}
```

61. DECISION RECORD

Example:

```
{
  "decision": "BUY",
  "product": "milk",
  "quantity": 1,
  "price": 64,
  "reason": "Predicted depletion within 24 hours",
  "confidence": 0.92,
  "authorization": "AUTO",
  "budgetImpact": 64
}
```

62. AGENT TOOL BOUNDARIES

The agent should not directly modify important state.

Instead:

```
AGENT
  ↓
REQUEST
  ↓
TOOL
  ↓
VALIDATION
  ↓
STATE CHANGE
```

For example:

Agent:
"Purchase milk."

Policy Tool:
"Authorized."

Budget Tool:
"Funds available."

Transaction Tool:
"Order permitted."

Commerce Tool:
"Order executed."

63. FAILURE HANDLING

The MVP should explicitly handle:

- Product unavailable
- Retailer unavailable
- Tool failure
- Order failure
- Low inventory confidence
- Budget conflict
- Policy conflict
- Ambiguous user intent
- Duplicate order
- Authentication failure

Example:

If the selected product becomes unavailable:

```
RETRY SEARCH
  ↓
ALTERNATIVE PRODUCT
  ↓
POLICY CHECK
  ↓
ASK / BUY
```

The agent should never silently assume that a failed transaction succeeded.

64. TESTING STRATEGY

The project will include deterministic test cases for:

Inventory

- Correct inventory estimate
- Low-confidence inventory
- No-purchase scenario

Intent

- Maggi
- Pasta
- Biryani for six
- Ambiguous meal request

Budget

- Within budget
- Above automatic limit
- Insufficient budget

Policy

- Allowed category
- Restricted category
- Autonomous mode disabled

Commerce

- Product found
- Product unavailable
- Cart creation
- Checkout failure

Agent

- Correct tool selection
- Correct decision
- Correct explanation

65. CORE TEST SCENARIOS

TEST 1

Input:

Milk predicted to run out tomorrow.

Expected:

BUY

TEST 2

Input:

Oil inventory sufficient.

Expected:

DO_NOT_BUY

TEST 3

Input:

Rice cost = ₹1,200
Auto limit = ₹500

Expected:

ASK_USER

TEST 4

Input:

"I want to make Maggi."

Inventory:

```
Oil = available
Spices = available
Maggi = missing
```

Expected:

```
BUY MAGGI ONLY
```

TEST 5

Input:

```
"I want to make pasta for four."
```

Expected:

```
Generate requirements
+
Reconcile inventory
+
Create missing-item shopping plan
```

TEST 6

Input:

```
Inventory confidence = 38%
```

Expected:

```
DO NOT AUTO-PURCHASE
ASK USER
```

66. EVALUATION METRICS

The project will measure:

Decision Accuracy

Percentage of scenarios where the agent chooses the correct action.

Policy Compliance

Percentage of transactions where the agent correctly follows authorization rules.

Inventory Decision Accuracy

Percentage of test cases where the agent correctly identifies:

- Buy
- Wait
- Do not buy

Intent Accuracy

Percentage of user requests correctly converted into requirements.

False Purchase Rate

Number of unnecessary purchases generated by the agent.

False Non-Purchase Rate

Number of necessary purchases incorrectly skipped.

User Intervention Rate

Percentage of scenarios requiring human involvement.

67. AGENT EVALUATION PHILOSOPHY

The project should optimize for:

CORRECT AUTONOMY

not:

MAXIMUM AUTONOMY

A good agent is not the agent that performs the most actions.

A good agent is the agent that performs the right actions and knows when it should stop.

68. TECHNICAL IMPLEMENTATION PRIORITIES

The six-week implementation will prioritize the following:

PRIORITY 1

Strands Agent

Must be genuine and central to the product.

PRIORITY 2

AgentCore Runtime

Deploy the working agent.

PRIORITY 3

AgentCore Memory

Persist household context.

PRIORITY 4

AgentCore Gateway

Expose household tools and commerce MCP.

PRIORITY 5

One real commerce integration

Use an official integration where possible.

PRIORITY 6

Deterministic policy engine

Protect autonomous actions.

PRIORITY 7

Confidence-aware inventory engine

Make uncertainty part of the decision process.

PRIORITY 8

Three excellent end-to-end demonstrations

Predictive purchase.

Do nothing.

Intent-to-purchase.

69. EXPLICITLY DEFERRED FEATURES

The following features are not part of the core six-week MVP:

- Multiple production retailer integrations
- Full multi-agent architecture
- Advanced household subscriptions
- Full payment abstraction
- Complex AgentCore Identity workflows
- Large-scale evaluation infrastructure
- Advanced event planning
- Full mobile application
- Complete household operating system
- Autonomous purchasing across every category
- Large notification infrastructure

These belong in the future roadmap.

70. SIX-WEEK DEVELOPMENT PLAN

WEEK 1 — FOUNDATION

Build:

- Strands agent
- Basic tools
- Household state model
- DynamoDB schema
- Basic UI
- Bedrock integration

WEEK 2 — HOUSEHOLD INTELLIGENCE

Build:

- Purchase history
- Consumption model
- Inventory estimation
- Confidence score
- Do-not-buy decision

WEEK 3 — INTENT ENGINE

Build:

- Natural-language intent
- Requirement generation
- Recipe/activity interpretation
- Inventory reconciliation
- Maggi workflow

WEEK 4 — COMMERCE

Build:

- Swiggy Instamart MCP integration
- Product search
- Cart
- Checkout flow where authorized
- Order state
- Retailer abstraction

WEEK 5 — AGENTCORE + SAFETY

Build:

- AgentCore Runtime
- AgentCore Memory
- AgentCore Gateway
- Budget engine
- Policy engine
- Approval workflow
- Audit log

WEEK 6 — TESTING + PRESENTATION

Complete:

- End-to-end tests
- Failure handling
- Demo environment
- Architecture diagram
- README
- Public repository
- Five-minute demo
- Presentation
- Builder.aws build story

71. REPOSITORY STRUCTURE

```
household-autopilot/  
|  
├── README.md  
├── LICENSE  
|  
├── agent/  
|   ├── household_agent.py  
|   ├── prompts/  
|   └── tools/  
|  
├── household/  
|   ├── profile.py  
|   ├── memory.py  
|   └── state.py  
|  
├── inventory/  
|   └── inventory_engine.py
```

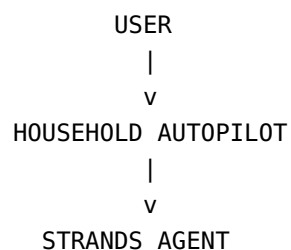
```

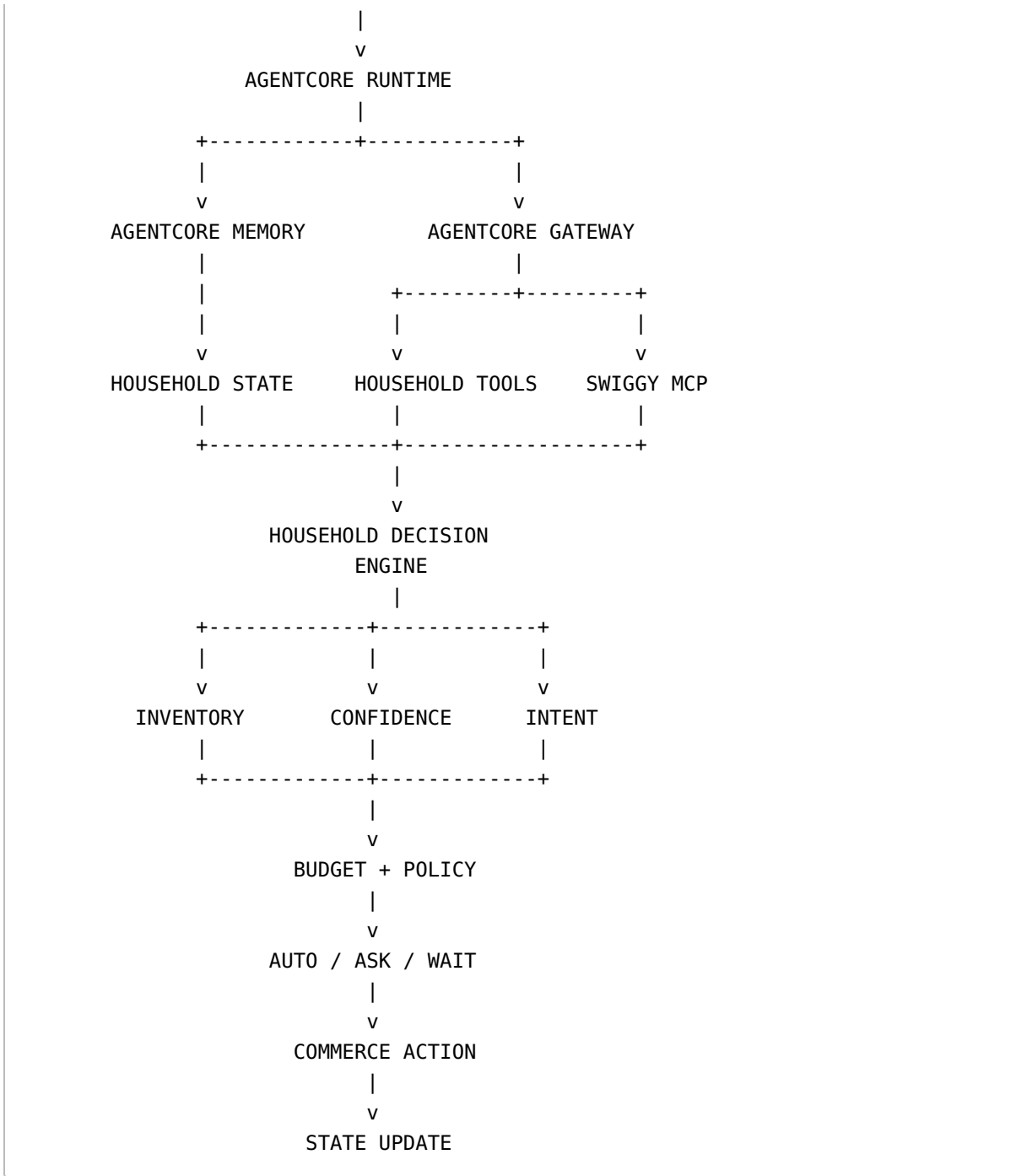
|   |   | consumption_engine.py
|   |   | confidence_engine.py
|
|   |--- intent/
|   |   |--- intent_engine.py
|   |   |--- requirement_engine.py
|
|   |--- decision/
|   |   |--- decision_engine.py
|   |   |--- budget_engine.py
|   |   |--- policy_engine.py
|
|   |--- commerce/
|   |   |--- interface.py
|   |   |--- swiggy_adapter.py
|   |   |--- mock_adapter.py
|
|   |--- workflows/
|   |   |--- background_check.py
|
|   |--- infrastructure/
|   |   |--- agentcore/
|   |   |--- lambda/
|   |   |--- eventbridge/
|
|   |--- frontend/
|
|   |--- tests/
|
|   |--- docs/
|   |   |--- architecture.md
|   |   |--- setup.md
|   |   |--- demo.md

```

72. ARCHITECTURE DIAGRAM FOR SUBMISSION

The final architecture diagram should communicate:





73. HACKATHON JUDGING STRATEGY

The hackathon evaluates five equally weighted criteria:

1. Technical Implementation
2. Design
3. Potential Impact

4. Creativity & Originality
5. Presentation.

The project will intentionally map its implementation to each criterion.

74. TECHNICAL IMPLEMENTATION

The project demonstrates genuine use of Strands Agents through:

- Multi-step tool use
- Persistent memory
- Background execution
- Intent reasoning
- Inventory reasoning
- Confidence-aware decisions
- Budget constraints
- Policy enforcement
- Commerce integration
- AgentCore deployment

The key technical demonstration is not the UI.

It is the agent trajectory:

```
REQUEST / EVENT
  ↓
RETRIEVE STATE
  ↓
REASON
  ↓
USE TOOLS
  ↓
CHECK CONSTRAINTS
  ↓
DECIDE
  ↓
ACT
  ↓
UPDATE STATE
```

This directly addresses the hackathon's requirement for a working, non-trivial Strands implementation.

75. DESIGN

The product should feel like a household control layer rather than a developer demo.

The user should be able to:

- Configure their household.
- Define their budget.
- Define autonomy rules.
- View inventory.
- View upcoming requirements.
- Approve exceptions.
- Understand decisions.
- Disable autonomy.

The hackathon explicitly evaluates whether the submission provides a complete and coherent product experience rather than only a technical proof of concept.

76. POTENTIAL IMPACT

The target users are households that repeatedly spend time managing routine purchases.

The impact is measured in:

- Reduced repetitive decision-making
- Reduced manual shopping work
- Reduced unnecessary purchases
- Better budget awareness
- Less household cognitive overhead

The system is designed around a real recurring problem rather than a purely technical demonstration.

77. CREATIVITY AND ORIGINALITY

The project does not claim that grocery automation itself is novel.

Instead, its originality is positioned around the combination of:

```
HOUSEHOLD STATE
+
UNCERTAINTY
+
USER INTENT
+
PANTRY RECONCILIATION
+
BUDGET
+
AUTHORIZATION
```

+
DECISION

The central design question is:

When should an autonomous household agent act, and when should it refuse to act?

This leads to the project's most important principle:

AUTONOMY $\neq$ AUTOMATION

AUTONOMY =
REASONING
+
CONSTRAINTS
+
CONFIDENCE
+
ACTION
+
RESTRAINT

78. PRESENTATION

The five-minute video should avoid presenting every feature.

It should tell one story.

The recommended story:

THE HOUSEHOLD HAS A PROBLEM
↓
THE AGENT OBSERVES IT
↓
THE AGENT PREDICTS A NEED
↓
THE AGENT ACTS
↓
THE AGENT REFUSES AN UNNECESSARY ACTION
↓
THE USER GIVES A NATURAL-LANGUAGE GOAL
↓
THE AGENT UNDERSTANDS THE GOAL

↓
THE AGENT CHECKS THE PANTRY
↓
THE AGENT BUYS ONLY WHAT IS MISSING

This demonstrates the entire agentic concept without requiring a long feature tour.

79. FIVE-MINUTE VIDEO PLAN

0:00–0:30 — PROBLEM

"Household shopping is not one transaction. It is a continuous series of decisions."

Show the manual workflow.

0:30–1:00 — PRODUCT

Introduce Household Autopilot.

"Instead of asking the user what to buy every time, the agent maintains a model of the household."

1:00–2:00 — PREDICTIVE ACTION

Show milk.

The agent detects depletion and automatically acts within policy.

2:00–2:30 — RESTRAINT

Show oil.

The agent determines that no purchase is required.

2:30–3:45 — INTENT

User:

"I want to make Maggi tonight."

Show:

Intent → Requirements → Inventory → Missing item → Retailer → Budget → Decision

3:45–4:30 — TECHNICAL ARCHITECTURE

Show:

Strands → AgentCore → Memory → Gateway → Tools → Commerce MCP → DynamoDB

4:30–5:00 — WHY IT MATTERS

"The goal is not to make shopping easier. The goal is to remove routine household shopping management from the user's responsibilities."

80. HACKATHON SUBMISSION REQUIREMENTS

The final submission will include:

- Project description
- Public GitHub repository
- Complete source code
- Assets
- Setup instructions
- MIT or Apache license
- README
- Architecture diagram
- Maximum five-minute demo video
- Working project demonstration
- Pitch covering:
 - Problem
 - Target audience
 - Why it matters
- AWS Builder ID
- Optional live demo

These are explicitly required or recommended by the hackathon rules.

81. LIVE DEMO

A live demo should be provided if the environment is stable enough for judging.

The hackathon states that a live demo link can strengthen the Technical Implementation score.

The live demo should include:

- Preconfigured demo household
- Working agent

- Working memory
- Working inventory model
- Working policy engine
- Working commerce integration or clearly labeled mock fallback
- Working approval flow

82. BUILDER.AWS BONUS

A public builder.aws post should document:

- Problem discovery
- Why existing approaches are insufficient
- Household Decision Engine
- Strands implementation
- AgentCore architecture
- Commerce MCP integration
- Inventory uncertainty
- Safety model
- Testing
- Lessons learned

The hackathon rules state that builder.aws content can contribute bonus points, with up to 0.6 additional points across qualifying posts.

83. FUTURE ROADMAP

The following features are intentionally future work rather than MVP requirements.

MULTI-RETAILER

Add:

- Zepto
- Amazon
- Other commerce providers

ADVANCED HOUSEHOLD MEMORY

Learn:

- Seasonal consumption
- Household events
- Long-term purchasing preferences
- Individual member preferences

ADVANCED INVENTORY

Integrate:

- Barcode scanning
- Receipt scanning
- Image-based pantry recognition
- User-confirmed inventory
- Smart-device integrations

ADVANCED AGENTCORE

Add:

- AgentCore Identity
- Advanced Evaluations
- More sophisticated Gateway policies
- Multi-agent orchestration

HOUSEHOLD OPERATING SYSTEM

Expand beyond groceries into:

- Cleaning supplies
- Toiletries
- Pet supplies
- Household maintenance
- Recurring subscriptions
- Other household errands

84. LONG-TERM VISION

The long-term goal is not to create the world's best grocery ordering agent.

The long-term goal is to create a:

HOUSEHOLD OPERATING SYSTEM

The system would maintain a continuously updated model of:

PEOPLE
+
INVENTORY
+
CONSUMPTION

+
PREFERENCES
+
BUDGET
+
ROUTINES
+
EVENTS
+
GOALS

The agent would then determine what actions are required to keep the household operating normally.

85. PRODUCT PRINCIPLE

The system should optimize:

HOUSEHOLD OUTCOME

not:

NUMBER OF ORDERS

and not:

MONEY SPENT

A successful agent may perform:

10 actions

or:

0 actions

depending on the household state.

86. SUCCESS METRICS

The MVP will measure:

Agent Performance

- Decision accuracy
- Tool selection accuracy
- Intent accuracy
- Successful task completion

Safety

- Unauthorized purchase rate
- Policy violation rate
- Duplicate purchase rate
- False autonomous-action rate

Inventory

- Inventory prediction accuracy
- Confidence calibration
- False purchase rate
- False non-purchase rate

User Experience

- Number of manual actions avoided
- Approval frequency
- User correction frequency
- Time saved

87. CORE TECHNICAL CONTRIBUTION

The primary technical contribution of the MVP is the implementation of a **confidence-aware household decision loop**.

The agent does not simply:

PREDICT → BUY

It performs:

```
PREDICT
↓
ESTIMATE CONFIDENCE
↓
CHECK NECESSITY
↓
CHECK AUTHORIZATION
↓
CHECK BUDGET
↓
DECIDE
```

This creates a safer and more realistic autonomous system.

88. CORE PRODUCT CONTRIBUTION

The primary product contribution is:

The household becomes the context, rather than the individual shopping request.

For example:

"I want to make Maggi."

is not interpreted as:

SEARCH FOR MAGGI

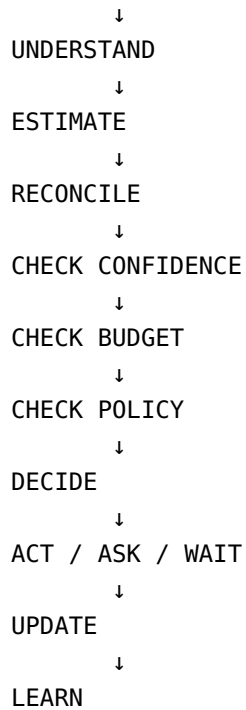
It is interpreted as:

WHAT DOES THIS HOUSEHOLD NEED
TO ACCOMPLISH THIS GOAL?

The agent then uses household state to determine the minimum required action.

89. FINAL PRODUCT LOOP

```
HOUSEHOLD STATE
+
USER GOAL
```



90. FINAL PROJECT DEFINITION

Household Autopilot is an uncertainty-aware autonomous AI agent that maintains a persistent model of a household and uses that model to make routine purchasing decisions.

It combines household memory, consumption patterns, estimated inventory, confidence scoring, natural-language intent understanding, requirement generation, inventory reconciliation, budget constraints, authorization policies and commerce tools.

The agent can operate proactively by predicting household needs and reactively by understanding user goals.

When the user says:

"I want to make Maggi."

the agent does not simply search for Maggi.

It determines what the activity requires, checks what is already available, identifies what is missing, evaluates the purchase against the household's budget and authorization policy, determines whether its information is reliable enough to act, and then either purchases the required item or asks the user.

When the household does not need something, the agent does nothing.

When the agent is uncertain, it asks.

When the agent is authorized and confident, it acts.

This creates an autonomous household system based not on maximum automation, but on **correct and bounded autonomy**.

91. FINAL ONE-LINE PITCH

"Household Autopilot is an AI agent that learns your household, understands what you need, knows when it is uncertain, and takes care of routine decisions within your rules."

92. FINAL HACKATHON PITCH

Household shopping is not a single task. It is a continuous responsibility.

Someone has to remember what is running out, estimate what is left, decide what is needed, compare options, check the budget, place the order and repeat the process every week.

Household Autopilot turns that responsibility into an autonomous agent.

It maintains a model of the household, predicts requirements, understands natural-language goals, reconciles those goals with estimated inventory, checks confidence, applies budget and authorization rules, and acts only when the decision is justified.

If the household needs milk, it can act.

If the household already has enough oil, it does nothing.

If the user says "I want to make Maggi," it understands the goal, checks the pantry, identifies what is missing and handles the purchase.

If the agent is uncertain, it asks instead of guessing.

The goal is not maximum automation.

The goal is:

THE RIGHT ACTION, AT THE RIGHT TIME, WITHIN THE USER'S RULES.

93. FINAL IMPLEMENTATION SCOPE

The six-week MVP will deliver:

1. Strands Agent
2. Amazon Bedrock
3. AgentCore Runtime
4. AgentCore Memory
5. AgentCore Gateway
6. DynamoDB household state
7. Inventory estimation
8. Consumption model
9. Confidence scoring
10. Budget engine
11. Policy engine
12. Natural-language intent
13. Requirement generation
14. Pantry reconciliation
15. One real commerce integration
16. Background EventBridge workflow
17. Auto / Ask / Do Nothing decisions
18. Audit trail
19. Three end-to-end demo scenarios
20. Public deployment/demo

Everything else is secondary.

94. FINAL ARCHITECTURAL PRINCIPLE

The system follows a strict separation:

LLM

=

UNDERSTAND + REASON + PLAN

DETERMINISTIC TOOLS

=

CALCULATE + VALIDATE + EXECUTE

MEMORY

=

REMEMBER

POLICY

=

AUTHORIZE

BUDGET
=
CONSTRAIN

CONFIDENCE
=
MEASURE UNCERTAINTY

COMMERCE
=
EXECUTE PURCHASE

AGENTCORE
=
RUN + CONNECT + REMEMBER

95. FINAL PROJECT STATEMENT

Household Autopilot is an autonomous household decision agent built with Strands Agents SDK and AWS services.

Its purpose is to remove repetitive household purchasing decisions from the user's daily responsibilities while preserving human control over important or uncertain actions.

The agent continuously reasons about:

WHAT DOES THE HOUSEHOLD HAVE?

WHAT DOES IT CONSUME?

WHAT WILL IT NEED?

WHAT DOES THE USER WANT TO DO?

WHAT IS MISSING?

HOW CONFIDENT IS THE AGENT?

WHAT DOES IT COST?

IS IT WITHIN THE BUDGET?

IS THE AGENT AUTHORIZED?

SHOULD IT ACT?

SHOULD IT ASK?

OR SHOULD IT DO NOTHING?

The project does not attempt to claim that autonomous grocery ordering is a new concept.

Instead, it demonstrates a deeper agentic problem:

How can an AI agent make reliable, explainable and bounded decisions about a household when the underlying state is incomplete and uncertain?

That is the problem Household Autopilot is designed to solve.